BitX AI Engine Architecture

Executive Summary

The BitX AI Engine is a locally-hosted intelligent automation platform that orchestrates 26 specialized AI agents across the full Software Development Lifecycle (SDLC). Unlike cloud-dependent AI solutions, BitX runs entirely on-premises using LM Studio, providing complete data sovereignty while delivering enterprise-grade agent capabilities.

The engine employs a **dual-model architecture** — a lightweight router model (Ministral 8B) for planning and coordination, and a powerful worker model (Qwen 32B Coder) for code generation and analysis. A 6-stage orchestration pipeline processes each request through intake, context gathering, planning, execution, validation, and finalization, ensuring consistent quality and safety guardrails.

Key differentiators:

- **100% local execution** — no data leaves the organization
- **26 purpose-built agents** spanning Discover, Design, Build, Test, and Deploy phases
- **Safety-first design** — 7 validators, path isolation, tool restrictions, and approval gates
- **RAG-powered context** — BM25 search over ingested documentation for informed decision-making

---

1. Architecture Overview

1.1 System Context

```
+-------------------+         +---------------------+         +------------------+
|                   |  HTTP   |                     |  HTTP   |                  |
|   React 19 UI     |-------->|   Fastify Backend   |-------->|   LM Studio      |
|   (Port 3000)     |  WS     |   (Port 4001)       |         |   (Port 1234)    |
|                   |<--------|                     |<--------|                  |
+-------------------+         +---------------------+         +------------------+
                                        |                       |
                                        v                       v
                              +------------------+    +------------------+
                              |   SQLite DB      |    |  Ministral 8B    |
                              |   (Drizzle ORM)  |    |  Qwen 32B Coder  |
                              +------------------+    +------------------+
```

Data flow: User submits a request via the React UI. The Fastify backend queues it through the Scheduler, which dispatches it to the Orchestrator. The Orchestrator drives the request through 6 stages, calling LM Studio for inference and the Tool Gateway for file operations. All state is persisted to SQLite.

1.2 Component Map

```
+-------------------------------------------------------------+
|                      BitX AI Engine                          |
|                                                              |
|  +------------------+    +------------------+               |
|  |   Scheduler      |    |   Orchestrator   |               |
|  |  Priority Queue   |--->|  6-Stage Pipeline |                |
|  |  Concurrency Ctrl |    |                  |                |
|  +------------------+    +--------+---------+               |
|                                   |                          |
|           +-----------+-----------+-----------+               |
|           |           |           |           |               |
|           v           v           v           v               |
|  +------------+ +----------+ +----------+ +------------+      |
```

```
|  | LLM Gateway| | Tool     | | Validator| | Profile    |        |
|  | Dual-Model | | Gateway  | | Engine   | | Loader     |        |
|  | Routing    | | 12 Tools | | 7 Rules  | | 26 Agents  |        |
|  +------------+ +----------+ +----------+ +------------+        |
|          |          |                                          |
|          v          v                                          |
|  +------------+ +----------+                                    |
|  | RAG Store  | | Run Store|                                    |
|  | BM25 Search| | SQLite   |                                    |
|  +------------+ +----------+                                    |
+-----------------------------------------------------------------+
```

1.3 Design Principles

Principle	Implementation
Local-First	All inference runs on LM Studio (localhost:1234). No external API calls.
Dual-Model Routing	Lightweight model for planning; powerful model for execution. Optimizes cost and speed.
Safety-by-Design	Path isolation, blocked file patterns, tool restrictions, approval gates — enforced at every layer.
Tenant Isolation	Queue throttling and data scoping per tenant.
Graceful Degradation	Worker fallback to router, RAG best-effort, retry with exponential backoff.
Observable	Every step, tool call, and validation result is persisted for audit and analysis.

---

2. Orchestrator Pipeline

The Orchestrator is the heart of the AI engine. It drives every agent run through a deterministic 6-stage pipeline.

2.1 Pipeline Overview

```
Request
   |
   v
+----------+    +-----------------+    +----------+    +-----------+    +------------+    +----------+
|  INTAKE  |--->| CONTEXT GATHERING|--->| PLANNING |--->| TOOL LOOP |--->| VALIDATION |--->| FINALIZE |
| (Router) |    | (Router/Worker)  |    | (Router) |    | (Worker)  |    |  (Engine)  |    | (Router) |
+----------+    +-----------------+    +----------+    +-----------+    +------------+    +----------+
```

2.2 Stage Details

Stage	Model	Purpose	Key Logic
1. Intake	Router or Worker (per profile)	Parse and understand the user request	Build system prompt from profile; send to LLM; capture initial understanding
2. Context Gathering	Router or Worker	Enrich with relevant knowledge	RAG search (top 5 results, score >= 0.05); up to 5 tool calls for file exploration
3. Planning	Router	Create step-by-step execution plan	Generate plan; save as `plan` artifact
4. Tool Loop	Worker or Router	Execute the plan using tools	Max 15 iterations; max 3 tool calls per iteration; `<done>` signal to exit

5. Validation	Engine (code)	Verify safety and quality	Run 7 validators; execute required tests; create approval if needed
6. Finalize	Router	Summarize and close	Generate summary; aggregate tokens/duration; transition to `completed`

2.3 State Machine

```
queued --> intake --> context_gathering --> planning --> tool_loop --> validation --> finalizing --> completed
                                                          |              |
                                                          |              v
                                                          |          approval --> finalizing
                                                          v
                                                        failed
```

Valid run statuses: `queued`, `intake`, `inventory`, `context_gathering`, `planning`, `tool_loop`, `validation`, `approval`, `finalizing`, `completed`, `failed`, `cancelled`

2.4 Error Recovery

- If any stage throws an exception, the run transitions to `failed` with the error message persisted
- The progress callback emits failure events to the frontend for real-time feedback
- The orchestrator does not retry internally — retry policy is managed at the profile level

---

3. LLM Gateway

The LLM Gateway provides a unified interface to LM Studio, abstracting model selection, retries, streaming, and health monitoring.

3.1 Dual-Model Architecture

Property	Router Model	Worker Model
Model	Ministral 8B Instruct	Qwen 2.5 Coder 32B Instruct
Context Window	16,384 tokens	8,192 tokens
Max Output Tokens	4,096	8,192
Temperature	0.3 (conservative)	0.2 (precise)
Timeout	60 seconds	120 seconds
Max Retries	2	3
Use Cases	Planning, routing, summarization, BA tasks	Code generation, testing, complex analysis

3.2 Model Discovery and Auto-Binding

On startup, the gateway calls `GET /v1/models` on LM Studio to discover loaded models. If the configured model IDs are found, they are used directly. Otherwise, auto-binding logic matches:

- **Router**: models containing "ministral" or "3b"
- **Worker**: models containing "devstral" or "24b", falling back to "coder" or "32b"

This allows zero-configuration startup — load any compatible models in LM Studio and the engine adapts.

3.3 Request Flow

```
ChatRequest --> getModelConfig(role) --> build request body --> POST /v1/chat/completions
                                                                          |
                                                              +-----------+-----------+
                                                              |                       |
                                                           Success                 Failure
                                                              |                       |
                                                        Parse response         Retry (exponential backoff)
                                                              |                       |
                                                        Return ChatResponse    Fallback to router (if worker)
```

3.4 Tool Call Parsing

The gateway parses tool calls from LLM output using XML tag extraction:

```
<tool_call>
{"tool": "read_file", "args": {"path": "src/index.ts"}}
</tool_call>
```

Multiple tool calls can appear in a single response. The `parseToolCalls()` method extracts all occurrences, while `extractTextBeforeToolCalls()` strips them to retrieve the reasoning text.

3.5 Health Monitoring

- Health checks run every 60 seconds
- Each model is checked against the discovered model list
- Health status is cached and exposed via `GET /local-agent/health`
- Status levels: `healthy` (all models available), `degraded` (some unavailable)

---

4. Scheduler

The Scheduler manages agent run concurrency, ensuring the system remains responsive under load.

4.1 Configuration

Parameter	Default	Description
`maxConcurrentRuns`	2	Maximum parallel agent runs
`maxQueueSize`	50	Maximum pending items in queue
`maxRunsPerTenant`	3	Per-tenant throttle (active + queued)
`queuePollIntervalMs`	2,000	Queue processing interval

4.2 Priority Queue

Runs are sorted by:

1. **Priority** (ascending — lower number = higher priority, default 10)
2. **Creation time** (FIFO within same priority)

The scheduler polls every 2 seconds. When a slot opens, the highest-priority waiting run is dispatched to the Orchestrator.

4.3 Queue Lifecycle

```
enqueue() --> validate limits --> add to queue --> sort by priority --> resolve with {runId, position}
                                                                |
```

```
processQueue()
      |
dequeue top item --> activeRuns.set() --> executeRun()
                                                |
                                        on complete/fail
                                                |
                                        activeRuns.delete()
                                                |
                                        processQueue() (next)
```

---

5. Tool Gateway

The Tool Gateway provides 12 tools that agents use to interact with the filesystem, run tests, and validate changes.

5.1 Tool Registry

Tool	Category	Read-Only	Description
`search_files`	code	Yes	Find files by name pattern (glob)
`read_file`	code	Yes	Read file contents (with optional line range)
`search_content`	code	Yes	Regex search across file contents
`write_file`	code	No	Create or overwrite a file
`apply_patch`	code	No	Apply find-and-replace patch to a file
`view_diff`	code	Yes	View git diff for a file or directory
`search_docs`	docs	Yes	Search markdown/yaml documentation
`read_schema`	docs	Yes	Read database or API schema files
`run_tests`	test	Yes	Execute a test suite (unit, integration, e2e, api, security, a11y, perf)
`read_test_output`	test	Yes	Read test result files
`validate_paths`	validation	Yes	Check paths are within allowed scope
`validate_diff_size`	validation	Yes	Check diff is within size limits

5.2 Execution Pipeline

```
Tool Request --> Check tool allowed (profile) --> Check forbidden (profile)
                                                        |
                                                Write operation?
                                                  /          \
                                               Yes           No
                                                |             |
                                          Validate path    Execute tool
                                                |             |
                                          Path safe?      Return result
                                            /      \
                                          Yes       No --> Error
                                           |
                                      Execute tool
```

```
                                        |
                                  Log + Return result
```

5.3 Path Safety

All write operations undergo path validation:

- **Traversal prevention**: Paths containing `..` are rejected
- **Forbidden paths**: Profile-specific forbidden patterns (e.g., `.env`, `.git/`, `node_modules/`)
- **Workspace boundary**: All resolved paths must stay within the configured working directory

5.4 Profile-Based Tool Filtering

Each agent profile defines:

- `allowedTools`: Whitelist (empty = all tools allowed)
- `forbiddenTools`: Blacklist (always enforced)

Read-only profiles (Discover/Design phase) are automatically forbidden from write tools (`write_file`, `apply_patch`).

---

6. Validator Engine

The Validator Engine enforces safety and quality rules after agent execution.

6.1 Seven Built-In Validators

Validator	Severity	What It Checks
`allowed_paths`	Error	All changed files are within workspace boundary and profile-allowed paths
`blocked_file_patterns`	Error	No changes to `.env`, `.git/`, `.ssh/`, credentials, secrets, keys, passwords, tokens
`required_tests`	Warning	Code-writing profiles have test suites configured
`diff_threshold`	Error	Diff size within limits (default: 500 lines, 20 files)
`approval_required`	Warning	Changes requiring human approval (always/risky/never threshold)
`tool_restrictions`	Error	All tool calls were within profile permissions; router profiles did not use write tools
`output_schema`	Warning	Artifacts have required structure (filePath for patches, valid JSON for reports)

6.2 Execution Strategy

1. **Core validators** always run: `blocked_file_patterns`, `tool_restrictions`
2. **Profile-required validators** run based on profile configuration
3. **Conditional validators** run based on context (e.g., `allowed_paths` only if files were changed)
4. **Summary** reports total errors and warnings; errors cause validation failure

6.3 Risk Assessment

The `approval_required` validator evaluates risk:

- **Always**: Profile requires approval for all changes
- **Risky**: Changes are flagged if diff exceeds 50% of limit, files match `migration|schema|config|security`, or more than 5 files changed
- **Never**: No approval needed

---

7. Run Persistence (Run Store)

All agent activity is persisted to SQLite via Drizzle ORM across 4 tables.

7.1 Data Model

```
+------------------+       +------------------+       +-------------------+
|   agent_runs     |       |   agent_steps    |       |  agent_artifacts  |
+------------------+       +------------------+       +-------------------+
| id (PK)          |<---+  | id (PK)          |<--+  | id (PK)           |
| tenant_id        |    |  | run_id (FK)      |   |  | run_id (FK)       |
| user_id (FK)     |    |  | step_order       |   |  | step_id (FK)      |
| agent_id (FK)    |    |  | stage            |   |  | type              |
| profile_id       |    |  | model            |   |  | name              |
| status           |    |  | role             |   |  | content           |
| router_model     |    |  | tool_name        |   |  | file_path         |
| worker_model     |    |  | tool_input (JSON)|   |  | metadata (JSON)   |
| initial_prompt   |    |  | tool_output(JSON)|   |  | created_at        |
| final_output     |    |  | tokens_used      |   |  +-------------------+
| error_message    |    |  | duration_ms      |   |
| total_tokens     |    |  | status           |   |  +-------------------+
| total_duration_ms|    |  | created_at       |   |  | agent_approvals   |
| metadata (JSON)  |    |  +------------------+   |  +-------------------+
| created_at       |    |                         |  | id (PK)           |
| started_at       |    +-------------------------+  | run_id (FK)       |
| completed_at     |                                 | step_id (FK)      |
+------------------+                                 | required_by       |
                                                     | reason            |
                                                     | status            |
                                                     | reviewer_id (FK)  |
                                                     | reviewer_note     |
                                                     +-------------------+
```

7.2 Artifact Types

Type	Description
`patch`	A code patch applied to a file
`diff`	A diff showing changes
`file_created`	A new file created by the agent
`file_modified`	An existing file modified
`test_result`	Test execution results
`report`	Structured analysis report (JSON)
`plan`	Execution plan from the planning stage
`brief`	Brief document or summary
`acceptance_criteria`	User story acceptance criteria
`summary`	Final run summary
`other`	Uncategorized artifact

7.3 Step Tracking

Every LLM call, tool execution, and validation check creates a step record with:

- Stage context (`intake` , `context_gathering` , `planning` , `tool_call` , `validation` , `finalize`)
- Model and role used
- Token consumption and duration
- Tool name, input, and output (for tool calls)
- Status and error information

---

8. Agent Profiles

8.1 Profile Organization (26 Agents)

SDLC Phase	Agents	Model Role	Count
Orchestrator	Product Manager	Router	1
Discover	Process Analyst, Business Analyst, User Journey	Router	3
Design	Solutions Architect, UX Designer, System Analyst, Technical Writer	Router	4
Build	Backend Dev, Frontend Dev, Full Stack Dev, DB Admin, Data Engineer, DevOps	Worker	6
Test	QA Lead, Unit Tester, Integration Tester, Functional Tester, Regression Tester, Performance Engineer, Accessibility Tester, Cross-Platform Tester, BA Validation Tester, UAT Coordinator	Router/Worker	10
Deploy	Release Manager, Security Analyst	Router/Worker	2

8.2 Profile Loading Strategy

1. **Local profiles first**: Load from `agents-local/profiles/{agent_name}/profile.json`
2. **Auto-convert Claude agents**: Load from `agents/{agent-name}.json` and convert to local format
3. **No overwrite**: Local profiles take precedence over auto-converted ones

The conversion maps Claude tool names to local equivalents, assigns model roles based on SDLC phase, configures boundaries, and generates system prompts from the agent definition.

8.3 Key Profile Fields

Field	Purpose
`modelRole`	`router` (lightweight tasks) or `worker` (code/test tasks)
`allowedTools` / `forbiddenTools`	Tool access control
`allowedPaths` / `forbiddenPaths`	Filesystem access boundaries
`approvalRequired` / `approvalThreshold`	Human-in-the-loop gates
`testRequired` / `requiredTestSuites`	Mandatory test execution
`maxDiffLines` / `maxFileChanges`	Change size limits
`boundaries`	Always/AskFirst/Never behavioral rules
`skills`	Slash commands the agent supports
`receives` / `produces`	Handoff chain definitions

---

9. Security Architecture

9.1 Defense in Depth

```
Layer 1: Tool Gateway        - Path safety, forbidden tools
Layer 2: Validator Engine     - Blocked patterns, workspace boundary
Layer 3: Profile Restrictions - Allowed/forbidden tools, paths, write permissions
Layer 4: Approval Gates       - Human review for risky changes
Layer 5: Tenant Isolation     - Per-tenant queue throttling
```

9.2 Blocked File Patterns

The system blocks access to sensitive files at the validator level:

- Environment files (`.env`)
- Version control (`.git/`)
- SSH keys (`.ssh/` , `id_rsa` , `.pem` , `.key`)
- Cloud credentials (`.aws/`)
- Secrets and tokens (`credentials` , `secrets.*` , `password` , `token`)

9.3 Read-Only Enforcement

Router-role profiles (Discover and Design phases) are:

- Forbidden from using `write_file` , `apply_patch` , `create_branch`
- Validated post-execution to ensure no write tools were used
- Limited to read and search operations only

---

10. API Surface

10.1 REST Endpoints

Method	Endpoint	Description
`GET`	`/local-agent/health`	System health, model status, queue stats
`GET`	`/local-agent/models`	Discovered and configured models
`POST`	`/local-agent/models/discover`	Trigger model discovery and auto-bind
`GET`	`/local-agent/profiles`	List all agent profiles (summary)
`GET`	`/local-agent/profiles/:id`	Full profile details
`POST`	`/local-agent/runs`	Submit a new agent run (queued)
`GET`	`/local-agent/runs`	List runs (admin=all, user=own)
`GET`	`/local-agent/runs/:id`	Run details + steps + artifacts + approvals
`GET`	`/local-agent/runs/:id/steps`	Steps for a run
`GET`	`/local-agent/runs/:id/artifacts`	Artifacts for a run
`GET`	`/local-agent/runs/:id/approvals`	Approvals for a run
`POST`	`/local-agent/approvals/:id/resolve`	Approve or reject a pending approval
`GET`	`/local-agent/queue`	Current queue status

DELETE	/local-agent/queue/:runId	Cancel a queued run
GET	/local-agent/tools	List all available tools
POST	/local-agent/route	Auto-select a profile for a prompt
GET	/local-agent/validators	List available validators
GET	/local-agent/eval/cases	List eval benchmark cases
POST	/local-agent/eval/run	Run full benchmark suite (async)
POST	/local-agent/eval/run-single	Run a single eval case

10.2 Authentication

All local-agent routes use **optional authentication**:

- If a valid JWT Bearer token is present, the user context is extracted
- If no token is present, routes still function (for local development)
- Admin users see all runs; regular users see only their own

10.3 Real-Time Updates

The Orchestrator emits progress events via callbacks at each stage transition:

```
{
  "runId": "abc123",
  "stage": "tool_loop",
  "message": "Executing tool: read_file",
  "data": { "tool": "read_file", "args": { "path": "src/index.ts" } }
}
```

These events propagate through WebSocket connections to the React frontend for live dashboard updates.

---

11. Eval Harness

The Eval Harness provides automated quality benchmarking across 11 test cases in 5 categories.

11.1 Categories

Category	Cases	Purpose
BA	2	Business analysis quality (user stories, gap analysis)
Engineering	2	Code generation quality (endpoints, bug fixes)
Testing	2	Test planning and execution quality
Repair	1	Bug fix and troubleshooting capability
Adversarial	4	Safety — .env access, path traversal, SQL injection, write from read-only

11.2 Scoring

Each case defines weighted criteria using 8 scoring types:

- `contains` / `not_contains` : Output text validation
- `tool_used` / `tool_not_used` : Tool usage verification

- `artifact_created` : Expected deliverable check
- `validation_passed` : Validator rule verification
- `status_equals` : Run outcome check
- `duration_under` : Performance threshold

Pass threshold: Weighted score >= 0.7 (70%)

Appendix A: Configuration Reference

Environment Variable	Default	Description
`LM_STUDIO_BASE_URL`	`http://localhost:1234`	LM Studio API endpoint
`LM_STUDIO_API_KEY`	`lm-studio`	API key for LM Studio
`LM_ROUTER_MODEL`	`ministral-8b-instruct-2410@q4_k_m:2`	Router model identifier
`LM_WORKER_MODEL`	`qwen2.5-coder-32b-instruct:2`	Worker model identifier
`SERVER_PORT`	`4001`	Fastify server port
`AGENT_WORKING_DIR`	`../product-hub`	Agent filesystem scope

Appendix B: Glossary

Term	Definition
Agent Profile	Configuration defining an agent's identity, capabilities, boundaries, and policies
Orchestrator	The 6-stage execution engine that drives agent runs
Router Model	Lightweight LLM used for planning and coordination
Worker Model	Powerful LLM used for code generation and complex tasks
Tool Gateway	Interface providing filesystem and test execution tools to agents
Validator	A safety or quality rule enforced after agent execution
RAG	Retrieval-Augmented Generation — enriching prompts with searched knowledge
BM25	Best Matching 25 — the ranking algorithm used for RAG search
Run	A single execution of an agent from queue to completion
Step	A discrete action within a run (LLM call, tool execution, validation)
Artifact	A deliverable produced during a run (code, report, plan, summary)

Cross-references: Learning Methodology | UI/UX Design | Dashboards | DDA Process Analyst